

CS 473 (Spring 2025)

Homework 6 Solutions

Problem 6.1: The original version of the smallest enclosing circle problem is very sensitive to so-called “outliers”: if a single point is corrupted, the optimal circle could change drastically. This motivates the following extension of the problem:

Given a set S of n points in 2D and an integer k , find the smallest circle that encloses at least $n - k$ points (i.e., we allow at most k points to be outside the circle).

In this question, you will design an efficient Monte Carlo algorithm to solve this new problem. You may assume that there are no degeneracies, i.e., no 4 points lie on a common circle.

- (a) (10 pts) Describe a naive algorithm that runs in $O(n^4)$ time.
- (b) (35 pts) Let C^* denote the optimal circle, let B^* denote the set of the (at most) 3 points on the boundary of C^* , and let Q^* denote the set of at most k points outside the circle. Form a random subset $R \subseteq S$ as follows: for each point $p \in S$, put p in R independently with probability $1/k$. (Consequently, $\mathbb{E}[|R|] = n/k$.)
Prove that the probability that $B^* \subseteq R \subseteq S - Q^*$ is at least $\Omega(1/k^3)$.
- (c) (40 pts) Describe a Monte Carlo algorithm that runs in $O(n)$ worst-case time and is correct with probability $\Omega(1/k^3)$. (Yes, this probability converges to 0, i.e., your algorithm is supposed to be wrong most of the time!)
[Hint: you may use the smallest enclosing circle algorithm from class as a subroutine; obviously, (b) is meant to help...]
- (d) (15 pts) Now, describe an efficient Monte Carlo algorithm that has error probability at most 0.01. Analyze the worst-case running time as a function of n and k . When k is small, it should be much faster than the algorithm in (a).

Solution:

- (a) By observations from class, the optimal circle is either the (unique) circumcircle through 3 of the points of S , or the (unique) circle that has 2 of the points of S as diameter. We can enumerate all of these $O(n^3 + n^2) = O(n^3)$ possible circles, test whether each of these circles encloses at least $n - k$ points in $O(n)$ time, and return the smallest circle that passes the test. The total time is $O(n^3 \cdot n) = O(n^4)$.

- (b) We have $B^* \subseteq R \subseteq S - Q^*$ iff every element in B^* is in R and every element in Q^* is not in R . The probability of this event is $(1/k)^{|B^*|}(1-1/k)^{|Q^*|}$, which is at least $(1/k)^3(1-1/k)^k$ since $|B^*| \leq 3$ and $|Q^*| \leq k$. To finish, observe that $\lim_{k \rightarrow \infty} (1-1/k)^k = e^{-1}$ is a constant and thus $(1-1/k)^k = \Omega(1)$.

(c) The algorithm:

1. $R = \emptyset$
2. for each $p \in S$ then
3. put p in R with probability $1/k$ (e.g., if $\text{random}(1, k)$ returns 1)
4. return the smallest circle C enclosing R

By the algorithm from class, line 4 takes $O(n)$ expected time. Lines 1–3 take $O(n)$ time. So, the total expected time is $O(n)$.

Error analysis:

Claim: If $B^* \subseteq R \subseteq S - Q^*$, then $C = C^*$.

Proof: Let r^* denote the radius of C^* . Since $R \subseteq S - Q^*$, then the radius of C is at most the radius of the smallest circle enclosing $S - Q^*$, which is r^* . On the other hand, since $R \supseteq B^*$, the radius of C is at least the radius of the smallest circle enclosing B^* , which is also r^* . Therefore, the radius of C is equal to r^* . Since C encloses B^* and the smallest circle enclosing B^* is unique, $C = C^*$.

Thus, if $B^* \subseteq R \subseteq S - Q^*$, our algorithm returns the correct answer. By part (b), this probability is at least $\Omega(1/k^3)$.

One minor technicality: the question asks for a worst-case time bound, not expected. We can use the following Las-Vegas-to-Monte-Carlo-conversion trick (briefly mentioned in class) to fix this: assuming that the algorithm takes cn expected number of steps, run the algorithm for at most $2cn$ steps, and if it does run to completion, return error. By Markov's inequality, the probability of success is now $1/2 \cdot \Omega(1/k^3) = \Omega(1/k^3)$.

(d) The algorithm (where c is a constant to be determined later):

1. for $i = 1, \dots, ck^3$ do {
2. run the algorithm in part (c) to get C_i
3. test whether C_i encloses at least $n - k$ points
- }
4. return the smallest circle that passes the test

Runtime: clearly $O(nk^3)$.

Error analysis: If one of the C_i 's is equal to C^* , then line 4 will correctly return C^* ,

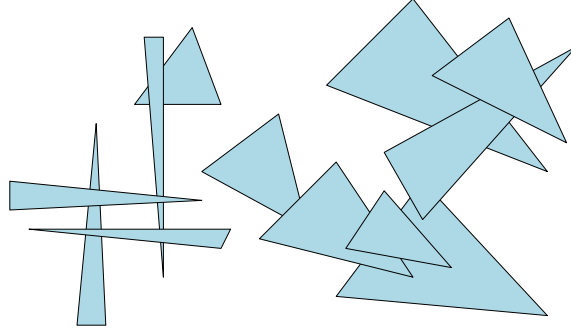
since C^* is the smallest circle that passes the test. Thus,

$$\begin{aligned}
\Pr\{\text{algorithm errs}\} &= \Pr\left(\bigcap_{i=1}^{ck^3} \{C_i \neq C^*\}\right) \\
&= \prod_{i=1}^{ck^3} \Pr\{C_i \neq C^*\} \quad \text{by independence} \\
&= \prod_{i=1}^{ck^3} (1 - b/k^3) \quad \text{by part (c) for some constant } b \\
&= (1 - b/k^3)^{ck^3} \\
&\leq \left(e^{-b/k^3}\right)^{ck^3} = e^{-bc} < 0.01
\end{aligned}$$

by setting $c = 5/b$, for example.

(Remark: the current fastest algorithm for this problem has running time near $O(n + k^2)$, ignoring logarithmic factors; see my paper.)

Problem 6.2: We are given n triangles $T_1, \dots, T_n$ in 2D. We want to compute the area of their union, i.e., $A = \text{area}(T_1 \cup \dots \cup T_n)$. (Note that the triangles may overlap, so A may not necessarily be equal to $\text{area}(T_1) + \dots + \text{area}(T_n)$.)



In this problem, we consider a randomized Monte Carlo algorithm to compute a good approximation of A . Let r be a parameter. Here is the pseudocode:

1. for $i = 1$ to n do
2. independently pick r random points $p_{i,1}, \dots, p_{i,r}$ from T_i
3. return $X = \sum_{i=1}^n |\{p_{i,1}, \dots, p_{i,r}\} \setminus (T_1 \cup \dots \cup T_{i-1})| \cdot \frac{\text{area}(T_i)}{r}$

(a) (30 pts) Prove that $\mathbb{E}[X] = A$.

[Hint: what is $\sum_{i=1}^n \text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1}))$?

- (b) (30 pts) Prove that $\text{Var}[X] \leq A^2/r$. ($\text{Var}[X]$ denotes the variance of X .)
- (c) (40 pts) By choosing r appropriately, show that the algorithm runs in $O(n^2)$ time and outputs a value X between $0.99A$ and $1.01A$ with probability at least 0.99.

[Note: you may assume that we can generate a random point in a triangle in $O(1)$ time, we can compute the area of a triangle in $O(1)$ time, and we can test whether a point is inside a triangle in $O(1)$ time. The above algorithm actually works for other types of objects besides triangles, and in dimension higher than 2.]

Solution:

- (a) For each $i \in \{1, \dots, n\}$ and $j \in \{1, \dots, r\}$, let $X_{i,j}$ be 1 if $p_{i,j} \notin T_1 \cup \dots \cup T_{i-1}$, and 0 else. Since $p_{i,j}$ is a random point in T_i ,

$$\mathbb{E}[X_{i,j}] = \Pr[p_{i,j} \notin T_1 \cup \dots \cup T_{i-1}] = \frac{\text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1}))}{\text{area}(T_i)}.$$

Now, $X = \sum_{i=1}^n \sum_{j=1}^r X_{i,j} \cdot \frac{\text{area}(T_i)}{r}$. By linearity of expectation,

$$\begin{aligned} \mathbb{E}[X] &= \sum_{i=1}^n \sum_{j=1}^r \mathbb{E}[X_{i,j}] \cdot \frac{\text{area}(T_i)}{r} \\ &= \sum_{i=1}^n r \cdot \frac{\text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1}))}{\text{area}(T_i)} \cdot \frac{\text{area}(T_i)}{r} \\ &= \sum_{i=1}^n \text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1})) \\ &= \text{area}(T_1 \cup \dots \cup T_n) = A, \end{aligned}$$

since the sets $T_i \setminus (T_1 \cup \dots \cup T_{i-1})$ are disjoint over all $i = 1, \dots, n$, and cover precisely $T_1 \cup \dots \cup T_n$ (if a point p belongs to $T_1 \cup \dots \cup T_n$, then by letting i be the smallest index for which $p \in T_i$, we would have $p \in T_i \setminus (T_1 \cup \dots \cup T_{i-1})$).

- (b) Let $X_{i,j}$ be the 0-1 random variable as defined in (a). Note that

$$\text{Var}[X_{i,j}] = \mathbb{E}[X_{i,j}^2] - \mathbb{E}[X_{i,j}]^2 \leq \mathbb{E}[X_{i,j}^2] = \mathbb{E}[X_{i,j}] = \frac{\text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1}))}{\text{area}(T_i)}.$$

Since the $X_{i,j}$'s are independent, by linearity of variance (and the fact that $\text{Var}[cY] =$

$c^2 \text{Var}[Y]$ for any fixed c),

$$\begin{aligned}
\text{Var}[X] &= \sum_{i=1}^n \sum_{j=1}^r \text{Var}[X_{ij}] \cdot \left(\frac{\text{area}(T_i)}{r} \right)^2 \\
&\leq \sum_{i=1}^n r \cdot \frac{\text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1}))}{\text{area}(T_i)} \cdot \left(\frac{\text{area}(T_i)}{r} \right)^2 \\
&\leq \sum_{i=1}^n \text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1})) \cdot \frac{\text{area}(T_i)}{r} \\
&\leq \sum_{i=1}^n \text{area}(T_i \setminus (T_1 \cup \dots \cup T_{i-1})) \cdot \frac{A}{r} \\
&\leq \text{area}(T_1 \cup \dots \cup T_n) \cdot \frac{A}{r} = \frac{A^2}{r}.
\end{aligned}$$

(c) The standard deviation of X is $\sigma = \sqrt{\text{Var}[X]} \leq A/\sqrt{r}$. By Chebyshev's inequality,

$$\Pr[|X - \mathbb{E}[X]| \geq c\sigma] \leq 1/c^2,$$

implying

$$\Pr[|X - A| \geq cA/\sqrt{r}] \leq 1/c^2.$$

Setting $c = 10$ and $r = 1000000$ gives

$$\Pr[|X - A| \geq 0.01A] \leq 0.01.$$

Thus, the algorithm outputs a value between $0.99A$ and $1.01A$ with probability at least 0.99 .

Checking whether one point $p_{i,j}$ is in or not in $T_1 \cup \dots \cup T_{i-1}$ requires $O(i) \leq O(n)$ time. Thus, the sum in line 3 requires $O(rn^2)$ time. Since we have chosen $r = 1000000$ (a constant), the run time is $O(n^2)$.

(*Remark.* Faster randomized algorithms (e.g., Karp, Luby, and Madras) are known that can achieve the same approximation guarantee with $O(n)$ running time.)

(*Remark.* Specifically for triangles in 2D, there is also a *deterministic* $O(n^2)$ -time algorithm that computes the area *exactly*, by computing the union explicitly using techniques from computational geometry. But this approach has much poorer run time in dimension 3 and higher, so the randomized approaches are better...)

Problem 6.3: We are given a (unweighted) bipartite graph G with n vertices and m edges. Suppose we have already computed a maximum matching M in G . Now suppose we delete a vertex v from G and all its incident edges. Let G' be the new graph (with $n - 1$ vertices). Describe how to efficiently compute a new maximum matching M' in G' . (Your algorithm should be faster than re-running a matching algorithm from scratch. Remember to prove correctness of your algorithm.)

Solution: As stated in the problem, we are given a maximum matching M of G . We remove a given vertex v from G to obtain a new (unweighted bipartite) graph G' , and wish to compute a maximum matching of G' .

Algorithm. Let M' be the result of removing v from the matching M . Note that M' is clearly a matching of G' .

Our algorithm is simply to run the Hungarian method on G' , but using M' (instead of the empty set) as the initial matching. When the Hungarian method stops, there is no augmenting path and so the output is a correct maximum matching of G' .

Analysis. Let M^{**} denote the maximum matching of G' . Since G' is a subgraph of G , the maximum matching size of G' is at most the maximum matching size of G , i.e., $|M^{**}| \leq |M|$.

Now, M' is the result of removing at most one edge of M (since at most one edge of M is incident to v by definition of matchings). Thus, $|M'| \geq |M| - 1$. It follows that $|M'| \geq |M^{**}| - 1$.

Each iteration increases the size of the current matching by one. Thus, the number of iterations is $O(|M^{**}| - |M'|) = O(1)$. Since each iteration takes $O(m)$ time by BFS/DFS as described in class, the total time is $O(1) \cdot O(m) = O(m)$.